

```
import tensorflow as tf
from tensorflow.keras import Sequential, Input
from tensorflow.keras.layers import Conv2D
from tensorflow.keras.preprocessing.image import ImageDataGenerator
```

```
tf.__version__
```

```
'2.19.0'
```

```
!unzip "/content/dataset_10 (1).zip"
```

```
unzip: cannot find or open /content/dataset_10 (1).zip, /content/dataset_10
```

```
from google.colab import drive
drive.mount('/content/drive', force_remount=True)
```

```
Mounted at /content/drive
```

```
!ls /content/drive/MyDrive/
```

```
'breast-cancer pratical 3 dataset.gsheat'
Classroom
'Colab Notebooks'
dataset
'dataset_10 (2).zip'
'I\'\'d card & adhar card '
iris_dataset.gsheat
Linear_regression_dataset.gsheat
'mall pratical 4 dataset.gsheat'
'pratical 1 dataset.gsheat'
'real_estate_price_size prat4 dataset.gsheat'
'real_estate_price_size SLR 5 datASET.gsheat'
'real_estate_price_size SLR.gsheat'
'real_estate_price_size_year MLR DATASET.gsheat'
SampleSuperstore_Profit_Report.gdoc
stock_data.gsheat
```

```
!unzip -q "/content/drive/MyDrive/your_file.zip" -d "/content/drive/MyDrive/"
```

```
unzip: cannot find or open /content/drive/MyDrive/your_file.zip, /content/dr
```

```
!ls /content/drive/MyDrive/
```

```
'breast-cancer pratical 3 dataset.gsheat'
Classroom
'Colab Notebooks'
dataset
'dataset_10 (2).zip'
'I\'\'d card & adhar card '
iris_dataset.gsheat
```

```

Linear_regression_dataset.gsheet
'mall pratical 4 dataset.gsheet'
'pratical 1 dataset.gsheet'
'real_estate_price_size prat4 dataset.gsheet'
'real_estate_price_size SLR 5 datASET.gsheet'
'real_estate_price_size SLR.gsheet'
'real_estate_price_size_year MLR DATASET.gsheet'
SampleSuperstore_Profit_Report.gdoc
stock_data.gsheet

```

```
!pip install gdown
```

```

Requirement already satisfied: gdown in /usr/local/lib/python3.12/dist-packag
Requirement already satisfied: beautifulsoup4 in /usr/local/lib/python3.12/di
Requirement already satisfied: filelock in /usr/local/lib/python3.12/dist-pac
Requirement already satisfied: requests[socks] in /usr/local/lib/python3.12/d
Requirement already satisfied: tqdm in /usr/local/lib/python3.12/dist-package
Requirement already satisfied: soupsieve>1.2 in /usr/local/lib/python3.12/dis
Requirement already satisfied: typing-extensions>=4.0.0 in /usr/local/lib/pyt
Requirement already satisfied: charset_normalizer<4,>=2 in /usr/local/lib/pyt
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.12/dist
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.1
Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.1
Requirement already satisfied: PySocks!=1.5.7,>=1.5.6 in /usr/local/lib/pytho

```

```

from google.colab import drive
drive.mount('/content/drive')

```

e already mounted at /content/drive; to attempt to forcibly remount, call driv

```

from google.colab import files
uploaded = files.upload()

```

dataset_10 (2).zip
dataset_10 (2).zip(application/x-zip-compressed) - 166677247 bytes, last modified: 4/19/2026
 - 100% done
 Saving dataset_10 (2).zip to dataset_10 (2) (1).zip

```
!unzip "/content/dataset_10 (2).zip"
```

```

inflating: dataset_10/val/chicken/OIP-UqdvRh_uUSgQyBXFiV4wBgHaEZ.jpeg
inflating: dataset_10/val/chicken/OIP-us-r-TgcJK9ldv95lP5G0gHaFh.jpeg
inflating: dataset_10/val/chicken/OIP-UwODZmRPDeLuTyMaKUEScwHaHx.jpeg
inflating: dataset_10/val/chicken/OIP-UzKhWda8800h3NiCXx7KJgHaE9.jpeg
inflating: dataset_10/val/chicken/OIP-v5b-HJ9QkIeTmEEKXFbP7QHaHX.jpeg
inflating: dataset_10/val/chicken/OIP-vL91T1VNt1TsDZk6xVv4sAHaEc.jpeg
inflating: dataset_10/val/chicken/OIP-V0JagJZmLSbf8yP5FgjbwHaGq.jpeg
inflating: dataset_10/val/chicken/OIP-vrTpppC8Ih0tDzPTdiJy4wHaFi.jpeg
inflating: dataset_10/val/chicken/OIP-vX0_5FfuGwtV5v-satQNPAAHaGl.jpeg
inflating: dataset_10/val/chicken/OIP-VyJaXomuB9_4YRnvB7aVIAHaFj.jpeg
inflating: dataset_10/val/chicken/OIP-w6LF82HwKckQVSfQ2VteMwHaIH.jpeg
inflating: dataset_10/val/chicken/OIP-w8gUUjb5TZ8wMc0i0t2y7wHaHa.jpeg
inflating: dataset_10/val/chicken/OIP-WaVxBatrIrUTfdqwhVcOeAHaFj.jpeg
inflating: dataset_10/val/chicken/OIP-wCWJ7E_dNacX1XUzXQ5V7gHaHX.jpeg
inflating: dataset_10/val/chicken/OIP-WDsZ9u6N8HJvHzgoiC13GwHaFj.jpeg
inflating: dataset_10/val/chicken/OIP-wgEv1UzjmkfEwIzoly4ZdgHaFj.jpeg
inflating: dataset_10/val/chicken/OIP-WKtZ1VQ7lqAp7YyYlMrAtQHaF7.jpeg
inflating: dataset_10/val/chicken/OIP-WPDZoZJ7_ShkGbkv3egZQAHaFj.jpeg
inflating: dataset_10/val/chicken/OIP-X3oXXSvr9QGm7wiR7_fsWgHaGQ.jpeg
inflating: dataset_10/val/chicken/OIP-x7AP_K0gzCu9A7DGjb6gVwHaFD.jpeg
inflating: dataset_10/val/chicken/OIP-XEZZ2ywikiY1wMwSyZ9gZQHaFj.jpeg
inflating: dataset_10/val/chicken/OIP-XgoEtJiF5hX2tE0-01avggFKC8.jpeg
inflating: dataset_10/val/chicken/OIP-xME8rmiVy0f-Uv5U0vLL9QHaF7.jpeg
inflating: dataset_10/val/chicken/OIP-XN7LiXpyJyLIiGuvkr1XoAHaJ4.jpeg
inflating: dataset_10/val/chicken/OIP-XpdPyhz2DsufGcb0AoZKtwHaFj.jpeg
inflating: dataset_10/val/chicken/OIP-xRYm-o2mbr81_0Syu24UUAHaF7.jpeg
inflating: dataset_10/val/chicken/OIP-xTWXtSCYSYh3qh2d7T8eFAHaH4.jpeg
inflating: dataset_10/val/chicken/OIP-xZd8hOnM0tfSFRZ-BiOg-QHaFj.jpeg
inflating: dataset_10/val/chicken/OIP-y1L0npUHLiy3ILzp5yB-ywHaFz.jpeg
inflating: dataset_10/val/chicken/OIP-YD_9SCYDEh4fdotRTvNPegHaFo.jpeg
inflating: dataset_10/val/chicken/OIP-yTNK11ZqJttOACZpdXgp9AHaFj.jpeg
inflating: dataset_10/val/chicken/OIP-z78q3dPlg8RQnZwbObEzlgHaEf.jpeg
inflating: dataset_10/val/chicken/OIP-ZeCCZito-7J6fz4CCB8qzgHaFj.jpeg
inflating: dataset_10/val/chicken/OIP-zKX1biRRtzx1F3AZbZLgpwHaFi.jpeg
inflating: dataset_10/val/chicken/OIP-zWLFuk_v3RUz-PhLxMEd1AHaFj.jpeg
inflating: dataset_10/val/chicken/OIP-_bfjOKkbF7szNndBXMHeHgHaEF.jpeg
inflating: dataset_10/val/chicken/OIP-_btU230Hny7Zl46R9t0qPwHaI5.jpeg
inflating: dataset_10/val/chicken/OIP-_Nd6-9eC_Mw4-2dBIU0lRgHaFj.jpeg
inflating: dataset_10/val/chicken/OIP-_shINE4hcbzellLi0_IkUwHaE8.jpeg
inflating: dataset_10/val/chicken/OIP-_TQiLInlBh8s5iLF5KpD5wHaFj.jpeg
inflating: dataset_10/val/chicken/OIP-_VBEevMZ5jJn5G2qG6GHVwHaE8.jpeg
inflating: dataset_10/val/chicken/OIP-_X5DG5EPJjcXGQNwNi874QHaGl.jpeg

```

```

train_datagen = ImageDataGenerator(
    rescale=1. / 255, #By dividing them by 255, you normalize them to the range [0, 1]
    shear_range=0.2, #image can be sheared by up to 20% of its width or height
    zoom_range=0.2, # image can be zoomed in or out by up to 20%.
    width_shift_range=0.2, # Randomly shift images horizontally by 20%
    height_shift_range=0.2, # Randomly shift images vertically by 20%
    rotation_range=20, # Randomly rotate images by 20 degrees
    horizontal_flip=True
)

training_set = train_datagen.flow_from_directory('/content/dataset_10/train',
                                                target_size = (64, 64),
                                                batch_size = 32,
                                                class_mode = 'binary')

```

Found 20938 images belonging to 10 classes.

```
test_datagen = ImageDataGenerator(rescale = 1./255)
test_set = test_datagen.flow_from_directory('/content/dataset_10/val',
                                           target_size = (64, 64),
                                           batch_size = 32,
                                           class_mode = 'binary')
```

Found 686 images belonging to 3 classes.

```
cnn = Sequential([
    Input(shape=(64, 64, 3)),
    Conv2D(filters=32, kernel_size=3, activation='relu')
])
```

```
cnn.add(tf.keras.layers.MaxPool2D(pool_size=2, strides=2))
```

```
cnn.add(tf.keras.layers.Conv2D(filters=64, kernel_size=3, activation='relu')
cnn.add(tf.keras.layers.MaxPool2D(pool_size=2, strides=2))
```

```
cnn.add(tf.keras.layers.Conv2D(filters=128, kernel_size=3, activation='relu')
cnn.add(tf.keras.layers.MaxPool2D(pool_size=2, strides=2))
```

```
cnn.add(tf.keras.layers.Flatten())
cnn.add(tf.keras.layers.Dense(units=128, activation='relu'))
```

```
cnn.add(tf.keras.layers.Dropout(0.5)) # 50% of neurons dropped during tra
```

```
cnn.add(tf.keras.layers.Dense(units=1, activation='sigmoid'))
```

```
from tensorflow.keras import Sequential, Input
from tensorflow.keras.layers import Conv2D, MaxPooling2D, Flatten, Dense, Dr
```

```
cnn = Sequential([
    Input(shape=(64, 64, 3)),

    # Block 1
    Conv2D(32, kernel_size=3, activation='relu'),
    MaxPooling2D(pool_size=2, strides=2),

    # Block 2
    Conv2D(64, kernel_size=3, activation='relu'),
    MaxPooling2D(pool_size=2, strides=2),

    # Block 3
```

```

Conv2D(128, kernel_size=3, activation='relu'),
MaxPooling2D(pool_size=2, strides=2),

# Classifier
Flatten(),
Dense(128, activation='relu'),
Dropout(0.5), # regularization
Dense(1, activation='sigmoid') # binary classification
])

cnn.summary()

cnn.compile(optimizer='adam',
            loss='binary_crossentropy',
            metrics=['accuracy'])

```

Model: "sequential_1"

Layer (type)	Output Shape	Param #
conv2d_3 (Conv2D)	(None, 62, 62, 32)	896
max_pooling2d_3 (MaxPooling2D)	(None, 31, 31, 32)	0
conv2d_4 (Conv2D)	(None, 29, 29, 64)	18,496
max_pooling2d_4 (MaxPooling2D)	(None, 14, 14, 64)	0
conv2d_5 (Conv2D)	(None, 12, 12, 128)	73,856
max_pooling2d_5 (MaxPooling2D)	(None, 6, 6, 128)	0
flatten_1 (Flatten)	(None, 4608)	0
dense_2 (Dense)	(None, 128)	589,952
dropout_1 (Dropout)	(None, 128)	0
dense_3 (Dense)	(None, 1)	129

Total params: 683,329 (2.61 MB)

Trainable params: 683,329 (2.61 MB)

```

import tensorflow as tf
from tensorflow.keras import Sequential, Input
from tensorflow.keras.layers import Conv2D, MaxPooling2D, Flatten, Dense, Dr
from tensorflow.keras.preprocessing.image import ImageDataGenerator

# ImageDataGenerator definitions
train_datagen = ImageDataGenerator(
    rescale=1. / 255,
    shear_range=0.2,
    zoom_range=0.2,
    width_shift_range=0.2,
    height_shift_range=0.2,
    rotation_range=20,
    horizontal_flip=True

```

```

)

training_set = train_datagen.flow_from_directory('/content/dataset_10/traini
                                                target_size = (64, 64),
                                                batch_size = 32,
                                                class_mode = 'categorical')

test_datagen = ImageDataGenerator(rescale = 1./255)
test_set = test_datagen.flow_from_directory('/content/dataset_10/val',
                                             target_size = (64, 64),
                                             batch_size = 32,
                                             class_mode = 'categorical') # Ch

cnn = Sequential([
    Input(shape=(64, 64, 3)),

    # Block 1
    Conv2D(32, kernel_size=3, activation='relu'),
    MaxPooling2D(pool_size=2, strides=2),

    # Block 2
    Conv2D(64, kernel_size=3, activation='relu'),
    MaxPooling2D(pool_size=2, strides=2),

    # Block 3
    Conv2D(128, kernel_size=3, activation='relu'),
    MaxPooling2D(pool_size=2, strides=2),

    # Classifier
    Flatten(),
    Dense(128, activation='relu'),
    Dropout(0.5), # regularization
    Dense(10, activation='softmax') # Changed units to 10 (number of classe
])

lr = 0.001 # Your desired learning rate
optimizer = tf.keras.optimizers.Adam(learning_rate=lr)

cnn.compile(optimizer = optimizer, loss = 'categorical_crossentropy', metric

"""### Training the CNN on the Training set and evaluating it on the Test se

from tensorflow.keras.callbacks import EarlyStopping

# Create the EarlyStopping callback
early_stop = EarlyStopping(
    monitor='val_accuracy',      # Metric to monitor
    patience=3,                  # Number of epochs with no improvement before
    restore_best_weights=True    # Roll back to best model
)

# Train the model with the callback
history = cnn.fit(
    x=training_set,
    validation_data=test_set,

```

```

epochs=5,                # Set a high max epoch
verbose=1,
callbacks=[early_stop]
)

```

Found 20938 images belonging to 10 classes.

Found 686 images belonging to 3 classes.

Epoch 1/5

655/655 ————— 0s 158ms/step - accuracy: 0.2166 - loss: 2.1492

ValueError Traceback (most recent call last)
 /tmp/ipykernel_5476/4166567612.py in <cell line: 0>()

```

65
66 # Train the model with the callback
---> 67 history = cnn.fit(
68     x=training_set,
69     validation_data=test_set,

_____ 1 frames _____
/usr/local/lib/python3.12/dist-packages/keras/src/backend/tensorflow/nn.py
in categorical_crossentropy(target, output, from_logits, axis)
    1164     for e1, e2 in zip(target.shape, output.shape):
    1165         if e1 is not None and e2 is not None and e1 != e2:
-> 1166             raise ValueError(
    1167                 "Arguments `target` and `output` must have the same
shape. "
    1168                 "Received: "

```

ValueError: Arguments `target` and `output` must have the same shape.
 Received: target.shape=(None, 3), output.shape=(None, 10)

Next steps: [Explain error](#)

```

import os

validation_path = '/content/dataset_10/val'
class_counts = {}

for class_name in os.listdir(validation_path):
    class_path = os.path.join(validation_path, class_name)
    if os.path.isdir(class_path):
        num_images = len(os.listdir(class_path))
        class_counts[class_name] = num_images

print("Number of images per class in validation set:")
for class_name, count in class_counts.items():
    print(f" {class_name}: {count} images")

# Also print the total number of classes found
print(f"Total number of classes found in validation set: {len(class_counts)}")

```

Number of images per class in validation set:
 butterfly: 211 images

```

chicken: 309 images
cat: 166 images
Total number of classes found in validation set: 3

```

```

import matplotlib.pyplot as plt

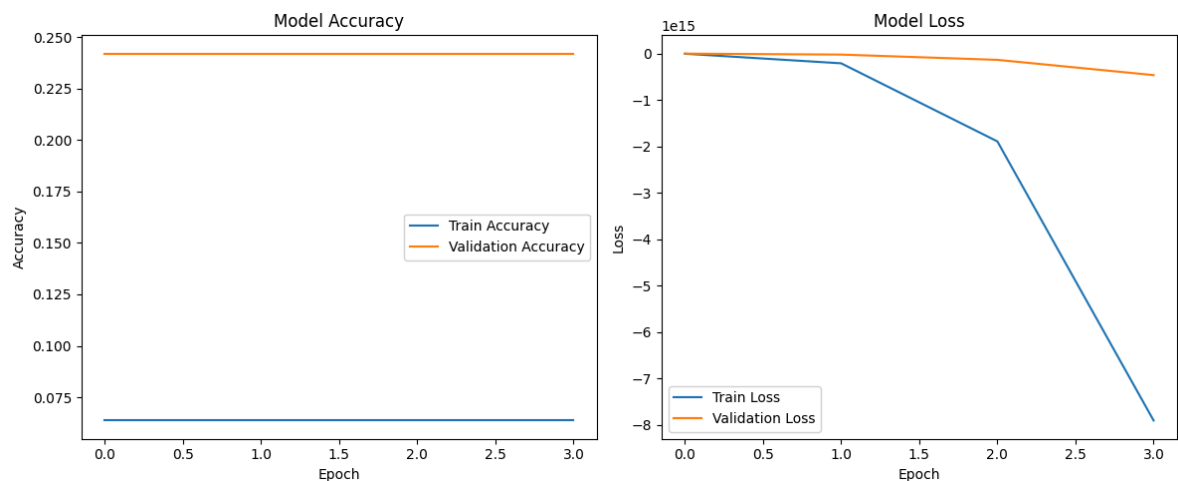
# Create a figure with 2 subplots (1 row, 2 columns)
fig, axs = plt.subplots(1, 2, figsize=(12, 5))

# Accuracy subplot
axs[0].plot(history.history['accuracy'], label='Train Accuracy')
axs[0].plot(history.history['val_accuracy'], label='Validation Accuracy')
axs[0].legend()
axs[0].set_title('Model Accuracy')
axs[0].set_xlabel('Epoch')
axs[0].set_ylabel('Accuracy')

# Loss subplot
axs[1].plot(history.history['loss'], label='Train Loss')
axs[1].plot(history.history['val_loss'], label='Validation Loss')
axs[1].legend()
axs[1].set_title('Model Loss')
axs[1].set_xlabel('Epoch')
axs[1].set_ylabel('Loss')

# Adjust layout and show
plt.tight_layout()
plt.show()

```



```

cnn.save("cnn.keras")

from keras.models import load_model
cnn = load_model('cnn.keras')

"""## Part 4 - Making a single prediction"""

import numpy as np
from tensorflow.keras.preprocessing import image

```

```
# Corrected path to an existing image for prediction
test_img = image.load_img('/content/dataset_10/val/cat/1002.jpeg', target_size=(256, 256))
test_img = image.img_to_array(test_img)
test_img = np.expand_dims(test_img, axis=0) / 255.0 # normalize

result = cnn.predict(test_img)

# The model was compiled for 10 classes with softmax, so a single binary class
# Instead, we should find the class with the highest probability
predicted_class_index = np.argmax(result[0])

# Get class names from the training_set generator to map index to class label
# You might need to re-run the cell defining training_set if this part fails
class_labels = list(training_set.class_indices.keys())
predicted_class_label = class_labels[predicted_class_index]

print(f"Predicted class: {predicted_class_label}")

"""## Well Done!"""
```

1/1  0s 77ms/step
 Predicted class: ragno
 '## Well Done!'

```
!ls /content/dataset_10/val/cat/
```

```
1002.jpeg
1005.jpeg
1014.jpeg
1033.jpeg
1043.jpeg
1047.jpeg
1076.jpeg
10.jpeg
1164.jpeg
1165.jpeg
1175.jpeg
1187.jpeg
1191.jpeg
1192.jpeg
1198.jpeg
119.jpeg
1207.jpeg
1216.jpeg
1219.jpeg
1222.jpeg
1230.jpeg
1250.jpeg
1269.jpeg
1277.jpeg
128.jpeg
1302.jpeg
1308.jpeg
1313.jpeg
1348.jpeg
1366.jpeg
```

1374.jpeg
139.jpeg
1428.jpeg
1460.jpeg
1469.jpeg
1476.jpeg
1483.jpeg
1484.jpeg
1487.jpeg
1512.jpeg
1525.jpeg
1541.jpeg
1542.jpeg
1545.jpeg
1566.jpeg
1593.jpeg
1594.jpeg
159.jpeg
1602.jpeg
162.jpeg
1633.jpeg
163.jpeg
1647.jpeg
1653.jpeg
1659.jpeg
165.jpeg
1662.jpeg